Diseño e Implementación de un Compilador x86-64 para un Subconjunto de C# con Optimización y Evaluación Comparativa

Mateo Ramírez, David Silva
CS3402 – Compiladores
Universidad de Ingeniería y Tecnología – UTEC
Lima, Perú

***Resumen*—Este informe presenta el diseño e implementación de un compilador en C++ para un subconjunto inspirado en C#. El compilador realiza análisis léxico, parsing descendente recursivo, construcción de AST, análisis semántico, optimizaciones básicas y generación de ensamblador x86-64 en sintaxis AT&T. Además, se evalúa experimentalmente frente a GCC, Clang/LLVM y .NET Native AOT mediante benchmarks equivalentes.**

I. Introducción y Arquitectura

El objetivo del proyecto fue construir un compilador funcional de extremo a extremo para un lenguaje con sintaxis similar a C#, orientado a la generación de código x86-64 ejecutable. La implementación se realizó manualmente en C++. El pipeline implementado es:

Fuente → Lexer → Parser → AST → Type Checker → Optimización → x86-64 → Ejecutable

Figura 1. Pipeline general del compilador.

Cuadro I
Componentes principales

Componente	Archivo principal
Tokens y lexer	`token.cpp`, `lexer.cpp`
Parser	`parser.cpp`
AST y Visitor	`ast.cpp`, `visitor.h`
Tabla de símbolos	`environment.h`
Análisis semántico	`typechecker.cpp`
Generación x86-64	`codegen.cpp`
Optimizaciones	`constant`, `sethi`, `peephole`

El backend genera ensamblador x86-64. Los valores de retorno se ubican en `%rax`; los primeros argumentos se pasan mediante `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8` y `%r9`. Las variables locales se almacenan en la pila mediante offsets relativos a `%rbp`.

II. Lenguaje Implementado

El lenguaje fuente corresponde a un subconjunto inspirado en C#. No se implementa el runtime completo de .NET, sino un conjunto controlado de características compilables directamente a x86-64.

Cuadro II
Características implementadas

Categoría	Soporte
Estructura	`class Program`, métodos `static`
Tipos básicos	`int`, `bool`, `char`, `string`, `void`
Variables	Declaración, asignación y scopes anidados
Constantes	`const` local con tipo explícito e inicializador obligatorio
Funciones	Parámetros, llamadas y retorno
Control	`if`, `while`, `for`, `do-while`, `break`
Structs	Campos y acceso con punto
Arreglos	Arreglos 1D y 2D
Strings	Literales, asignación e impresión
Punteros	`&`, `*`, `int*`
Inferencia	`var` local
Casts	Conversiones explícitas simples
Lambdas	Lambdas simples sin closures

II-A. Gramática resumida

La gramática fue diseñada para ser procesada mediante parser descendente recursivo. Las expresiones se separan por niveles de precedencia para evitar ambigüedades.

Listing 1. Gramática resumida del subconjunto

```
Program     ::= ClassDecl+
ClassDecl   ::= "class" id "{" MemberDecl* "}"
MemberDecl  ::= StructDecl | MethodDecl
StructDecl  ::= "struct" id "{" FieldDecl* "}"
MethodDecl  ::= "static" Type id "(" ParamList? ")" Block
ParamList   ::= Param ("," Param)*
Param       ::= Type id
Block       ::= "{" Stmt* "}"

Stmt        ::= VarDecl | ConstDecl | Assign | If | While |
    For
              | DoWhile | Return | Break | Print | Block
ConstDecl   ::= "const" Type id "=" Exp ";"

Exp         ::= OrExp
OrExp       ::= AndExp ("||" AndExp)*
AndExp      ::= EqExp ("&&" EqExp)*
EqExp       ::= RelExp (("==" | "!=") RelExp)*
RelExp      ::= AddExp (("<" | "<=" | ">" | ">=") AddExp)*
AddExp      ::= MulExp (("+" | "-") MulExp)*
MulExp      ::= UnaryExp (("*" | "/" | "%") UnaryExp)*
UnaryExp    ::= Primary | "!" UnaryExp | "-" UnaryExp
              | "&" UnaryExp | "*" UnaryExp
Primary     ::= id | number | string | char | bool
              | call | access | "(" Exp ")"
```

III. Semántica, x86 y Optimización

El análisis semántico recorre el AST mediante el patrón Visitor y utiliza una tabla de símbolos con scopes anidados.

Se registran variables, funciones y structs, validando que cada identificador sea usado dentro de su alcance y con un tipo compatible.

También se implementó soporte para constantes locales mediante `const`. Una declaración constante requiere tipo explícito e inicializador obligatorio, por ejemplo `const int x = 10;`. Durante el análisis semántico, el compilador registra estas variables como constantes dentro de su scope y rechaza asignaciones posteriores al mismo identificador.

Listing 2. Constante local soportada

```
class Program {
    static int Main() {
        const int edad = 20;
        const string nombre = "Luis";

        Console.WriteLine(nombre);
        Console.WriteLine(edad);

        return 0;
    }
}
```

Cuadro III
VALIDACIONES SEMÁNTICAS

Validación	Ejemplo detectado
Variable no declarada	`x = 5;`
Asignación incompatible	`int x = true;`
Reasignación de constante	`const int x = 10; x = 20;`
Condición no booleana	`if (5) {...}`
Aridad incorrecta	`f(1,2,3);`
Retorno incompatible	`return "hola";` en función `int`
`break` inválido	`break;` fuera de un bucle
Campo inexistente	`p.z`

Durante la generación de código, cada función emite prólogo, reserva de pila, cuerpo y epílogo. El siguiente fragmento muestra el estilo de ensamblador generado.

Listing 3. Fragmento x86-64 generado

```
.globl main
main:
    pushq %rbp
    movq %rsp, %rbp
    subq $32, %rsp

    movq $5, %rax
    movq %rax, -8(%rbp)

.end_main:
    leave
    ret
```

Las optimizaciones implementadas son plegado de constantes, etiquetado Sethi-Ullman, strength reduction, peephole y eliminación simple de código muerto. Por ejemplo, una expresión como `2 + 3 * 4` puede evaluarse en compilación y reemplazarse por `14`. Además, Sethi-Ullman permite ordenar la evaluación de subexpresiones para reducir operaciones auxiliares sobre la pila.

IV. Evaluación Experimental

La evaluación comparó el compilador propio contra GCC, Clang/LLVM y .NET Native AOT. Cada benchmark fue escrito con algoritmo, salida y ancho de entero equivalentes. Como el `int` del compilador se implementa como entero de 64 bits, las versiones C usan `int64_t`/`uint64_t` y las versiones .NET usan `long` cuando corresponde.

Cuadro IV
CONFIGURACIONES COMPARADAS

Config.	Herramienta
`mine-noopt`	Compilador propio `-no-opt` + `g++ -no-pie`
`mine-opt`	Compilador propio `-opt` + `g++ -no-pie`
`gcc-O0`	`gcc -O0`
`gcc-O2`	`gcc -O2`
`clang-O2`	`clang -O2`
`dotnet-aot`	`dotnet publish -p:PublishAot=true`

Se midió tiempo de compilación, tiempo de ejecución, líneas de ensamblador generado, tamaño del ejecutable y salida del programa. Cada salida fue verificada para asegurar equivalencia funcional entre configuraciones.

Cuadro V
BENCHMARKS UTILIZADOS

Benchmark	Característica	Entrada
`fib`	Recursión y llamadas	`Fib(35)`
`hashloop`	Bucle aritmético apretado	10^8 iteraciones
`sieve`	Arreglo 1D y saltos	$2 \cdot 10^6$
`matmul`	Arreglos 2D y bucles anidados	200×200

El entorno utilizado fue un contenedor Docker `mcr.microsoft.com/dotnet/sdk:8.0` sobre plataforma `linux/amd64`. El host físico fue Apple Silicon, por lo que los binarios x86-64 se ejecutaron bajo emulación. Por ello, los tiempos absolutos no representan hardware x86 nativo, pero las comparaciones relativas son válidas porque todas las configuraciones se ejecutaron bajo la misma emulación.

IV-A. Resultados

Cuadro VII
LÍNEAS DE ENSAMBLADOR GENERADO

Benchmark	mine-noopt	mine-opt
`fib`	89	83
`hashloop`	68	65
`sieve`	162	159
`matmul`	364	361

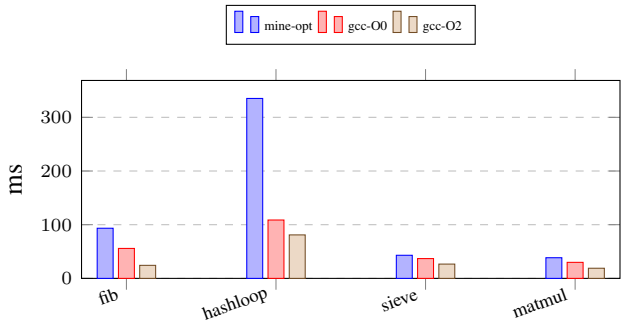


Figura 2. Tiempo de ejecución de `mine-opt` frente a GCC.

Cuadro VI
TIEMPO DE EJECUCIÓN EN MILISEGUNDOS, MEJOR DE 5 CORRIDAS

Benchmark	mine-noopt	mine-opt	gcc-O0	gcc-O2	clang-O2	dotnet-aot
`fib`	94.96	93.40	55.79	24.28	35.48	57.88
`hashloop`	327.58	335.16	108.73	80.99	25.45	96.71
`sieve`	42.06	43.04	36.89	26.62	32.60	41.71
`matmul`	38.14	38.48	29.86	18.88	17.42	41.76

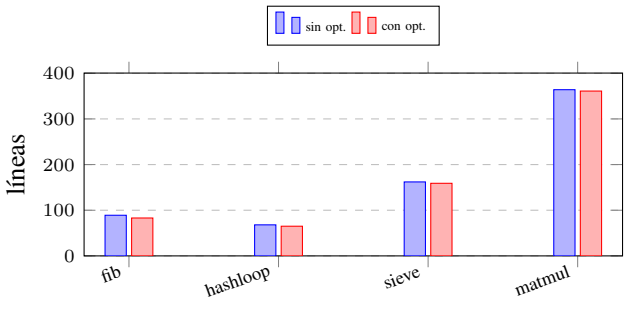


Figura 3. Reducción de líneas de ensamblador con `-opt`.

V. ANÁLISIS

El compilador genera código correcto y funcional para los cuatro benchmarks, con salidas idénticas frente a GCC, Clang y .NET Native AOT. En ejecución, `mine-opt` es entre 1.6 y 4.1 veces más lento que `gcc-O2` en los casos evaluados. El peor caso es `hashloop`, porque es un bucle apretado con dependencia de datos donde la falta de asignación global de registros produce mucho tráfico a memoria.

El efecto de `-opt` en tiempo de ejecución es marginal: por ejemplo, `fib` pasa de 94.96 ms a 93.40 ms, mientras que `sieve` pasa de 42.06 ms a 43.04 ms. Esto indica que el impacto temporal está dentro del ruido para estos benchmarks. Sin embargo, el efecto sí es visible en el tamaño del ensamblador: `fib` baja de 89 a 83 líneas, `hashloop` de 68 a 65, `sieve` de 162 a 159 y `matmul` de 364 a 361. La interpretación es que las optimizaciones implementadas son locales; no atacan todavía el cuello de botella principal, que es la falta de asignación de registros y optimizaciones de bucle.

En tiempo de compilación, el compilador propio aparece más rápido que GCC, Clang y .NET Native AOT, con tiempos cercanos a 103–108 ms. Esta comparación debe interpretarse con cuidado: el lenguaje aceptado es un subconjunto reducido y el tiempo está dominado por el ensamblado y enlace con `g++`, no por el frontend propio.

VI. CONCLUSIONES

El proyecto implementa un flujo completo de compilación: análisis léxico, parser descendente recursivo, AST, análisis semántico, optimización y generación de código x86-64 ejecutable. La evaluación experimental demuestra que el compilador produce resultados correctos frente a compiladores comerciales y que sus optimizaciones reducen consistentemente el tamaño del ensamblador generado.

La brecha frente a `gcc-O2` y `clang-O2` se explica principalmente por la ausencia de asignación global de registros, LICM, desenrollado de bucles, vectorización e inlining avanzado. Como trabajo futuro, las mejoras más importantes serían implementar asignación de registros, optimizaciones sobre CFG y benchmarks adicionales que permitan aislar mejor el efecto de cada optimización.

REFERENCIAS

[1] A. V. Aho, M. S. Lam, R. Sethi y J. D. Ullman, *Compilers: Principles, Techniques, and Tools*. Pearson, 2006.

[2] System V Application Binary Interface, AMD64 Architecture Processor Supplement.

[3] GNU Compiler Collection, “GCC Documentation”. Disponible en: https://gcc.gnu.org/onlinedocs/